

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



Hệ thống điểm danh nhân viên sử dụng nhận diện khuôn mặt

Nguyễn Hữu Duy

Mã học phần: MAT3508
Học kỳ 1, Năm học 2025-2026

Thông tin Dự án

Học phần: MAT3508 – Nhập môn Trí tuệ Nhân tạo
Học kỳ: Học kỳ 1, Năm học 2025-2026
Trường: VNU-HUS (Đại học Quốc gia Hà Nội – Trường Đại học Khoa học Tự nhiên)
Tên dự án: [Tên dự án của bạn]
Ngày nộp: [Ngày nộp] (ví dụ: 30/06/2025)
Báo cáo PDF: Liên kết tới báo cáo PDF trong kho GitHub
Slide thuyết trình: Liên kết tới slide thuyết trình trong kho GitHub
Kho GitHub: <https://github.com/23001853-wq/Employee-Attendance-System-Using-Camera.git>

Thành viên nhóm

Họ tên	Mã sinh viên	Tên GitHub
(Nguyễn Hữu Duy)	(23001853)	(23001853-wq)

Mục lục

1	Giới thiệu	4
1.1	Tóm tắt	4
1.2	Bài toán đặt ra	4
1.3	Thách thức	4
2	Phương pháp & Triển khai	5
2.1	Phương pháp	5
2.1.1	Thu thập dữ liệu (Data Collection)	5
2.1.2	Phát hiện và cắt khuôn mặt (Face Detection)	6
2.1.3	Tiền xử lý và Tăng cường dữ liệu (Preprocessing & Augmentation)	8
2.1.4	Trích xuất Đặc trưng với MobileNetV2	9
2.1.5	So sánh Thuật toán Phân loại	11
2.2	Triển khai	12
2.2.1	Xử lý và Chuẩn bị Dữ liệu	12
2.2.2	Huấn luyện và Đánh giá Mô hình	12
2.2.3	Triển khai Hệ thống Nhận diện	13
3	Kết quả & Phân tích	15
3.1	Mình họa dữ liệu	15
3.2	So sánh thuật toán	15
3.2.1	Phân tích tổng hợp	16
3.2.2	Ma trận nhầm lẫn - SVM (RBF Kernel)	18
3.3	Kết quả nhận diện thực tế với SVM	19
3.4	Hệ thống Web ứng dụng điểm danh	21
4	Kết luận	22
4.1	Tổng kết kết quả đạt được	22
4.2	Đóng góp của đề tài	23
4.3	Hạn chế của hệ thống	23
4.4	Hướng phát triển	23
A	Phụ lục	26
A.1	Hướng dẫn sử dụng các script chính	26
A.2	Mã Python trích xuất đặc trưng	26
A.3	Kết quả thử nghiệm chi tiết	27
A.4	Ghi chú quan trọng	27

Chương 1

Giới thiệu

1.1 Tóm tắt

Hệ thống điểm danh tự động dựa trên nhận diện khuôn mặt sử dụng học sâu (Deep Learning) được phát triển nhằm thay thế phương pháp điểm danh thủ công, nâng cao độ chính xác và tự động hóa. Dữ liệu khuôn mặt được thu thập qua webcam, tiền xử lý và lưu trữ, trước khi trích xuất đặc trưng bằng mạng MobileNetV2. Các vector đặc trưng 1280 chiều sau đó được **chuẩn hóa L2** để đảm bảo độ ổn định và độ chính xác khi đưa vào các thuật toán phân loại.

Trước khi trích xuất vector, hệ thống sử dụng thuật toán Haar Cascade của OpenCV để phát hiện và cắt vùng khuôn mặt, giúp xử lý tốt các điều kiện ánh sáng khác nhau và đảm bảo vector đầu vào chỉ chứa khuôn mặt. Các thuật toán phân loại được áp dụng gồm: SVM, KNN, Random Forest (học có giám sát) và Cosine Similarity, L2 Distance (học không giám sát), từ đó so sánh để lựa chọn giải pháp tối ưu.

Kết quả thực nghiệm cho thấy SVM (RBF Kernel) kết hợp chuẩn hóa L2 đạt độ chính xác cao nhất, nhận diện ổn định, thời gian xử lý nhanh và phù hợp triển khai thực tế. Hệ thống còn có giao diện **Streamlit** thân thiện, tích hợp cơ sở dữ liệu **SQL Server** để quản lý thông tin nhân viên và lịch sử điểm danh, đồng thời hỗ trợ tăng cường dữ liệu tự động nhằm cải thiện hiệu quả nhận diện.

1.2 Bài toán đặt ra

Điểm danh truyền thống bằng giấy tờ hoặc thẻ từ dễ gian lận, tốn thời gian và không phù hợp với quá trình chuyển đổi số. Bài toán đặt ra là xây dựng hệ thống điểm danh tự động, nhận diện chính xác khuôn mặt nhân viên trong điều kiện thực tế, đảm bảo an ninh, dễ sử dụng và có khả năng mở rộng.

1.3 Thách thức

Hệ thống cần giải quyết các thách thức sau:

1. **Đảm bảo độ chính xác cao với dữ liệu nhỏ:** Số lượng ảnh mỗi nhân viên ít, cần thuật toán học sâu nhẹ kết hợp SVM/KNN/RF để đạt hiệu quả mà không quá khớp.
2. **Xử lý đa dạng khuôn mặt và điều kiện ánh sáng:** Haar Cascade phát hiện khuôn mặt trên ảnh xám để tăng độ chính xác, sau đó cắt và lấy lại vùng mặt từ ảnh gốc màu trước khi trích xuất vector đặc trưng.
3. **Chuẩn hóa L2 và tối ưu tốc độ nhận diện:** Vector 1280 chiều từ MobileNetV2 được chuẩn hóa L2 và đưa vào SVM cho phép nhận diện nhanh, đủ cho ứng dụng real-time.
4. **So sánh nhiều thuật toán:** Học có giám sát và không giám sát để chọn mô hình tối ưu về độ chính xác, tốc độ và khả năng mở rộng.
5. **Quản lý check-in/check-out và cơ sở dữ liệu:** Sau khi nhận diện, hệ thống ghi nhận thời gian vào/ra nhân viên vào SQL Server, hỗ trợ tra cứu lịch sử và báo cáo.
6. **Khả năng mở rộng:** Thiết kế để mở rộng khi số lượng nhân viên tăng hoặc dữ liệu lớn hơn, dễ tích hợp các thuật toán Deep Learning nâng cao như ArcFace.

Chương 2

Phương pháp & Triển khai

2.1 Phương pháp

Pipeline nhận diện khuôn mặt được xây dựng dựa trên quy trình xử lý ảnh tiêu chuẩn, bao gồm ba giai đoạn chính: Thu thập dữ liệu đầu vào, Phát hiện khuôn mặt (Face Detection) và Tiền xử lý dữ liệu (Preprocessing).

2.1.1 Thu thập dữ liệu (Data Collection)

Dữ liệu đầu vào đóng vai trò quyết định đến độ chính xác của mô hình. Hệ thống hỗ trợ hai phương thức thu thập dữ liệu:

1. Thu thập trực tiếp qua Webcam:

- Hệ thống kích hoạt camera và tự động chụp liên tục khuôn mặt người dùng trong thời gian thực.
- Mỗi nhân viên sẽ được chụp khoảng 50 mẫu ảnh ở các góc độ và điều kiện ánh sáng khác nhau.
- Dữ liệu được gán nhãn với ID định danh (khóa chính) để phục vụ quá trình huấn luyện.



Hình 2.1: Giao diện thu thập dữ liệu trực tiếp từ Webcam - nhân viên nhìn thẳng vào camera để chụp nhiều góc độ khác nhau.

2. Thu thập từ nguồn ảnh có sẵn (Static Images):

- Đối với các nhân viên không thể có mặt trực tiếp hoặc dữ liệu người nổi tiếng, hệ thống cho phép nhập dữ liệu từ kho ảnh tĩnh.
- **Quy trình:** Tập hợp ảnh chân dung nhân viên (khuyến khích chọn ảnh đa dạng bối cảnh). Hệ thống sẽ quét thư mục, sử dụng thuật toán Haar Cascade để cắt lấy khuôn mặt và lưu vào tập dữ liệu huấn luyện.



Hình 2.2: Ví dụ thu thập dữ liệu từ ảnh tĩnh (Elon Musk) - ảnh chân dung.

2.1.2 Phát hiện và cắt khuôn mặt (Face Detection)

Để xác định vị trí khuôn mặt trong ảnh và loại bỏ nền, hệ thống sử dụng thuật toán **Viola-Jones (Haar Cascade)**. Đây là phương pháp máy học có tốc độ xử lý cực nhanh và độ chính xác cao, hoạt động dựa trên các thành phần cốt lõi sau:

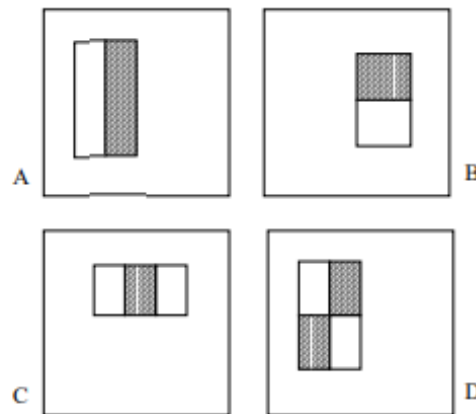
Đặc trưng Haar-like (Haar-like Features)

Thuật toán Viola-Jones sử dụng các đặc trưng Haar-like thay vì xử lý trực tiếp giá trị pixel. Những đặc trưng này được thiết kế để nắm bắt các đặc điểm cấu trúc quan trọng của khuôn mặt:

- **Đặc trưng 2-rectangle:** Phát hiện các cạnh và biên giới sáng-tối (ví dụ: hốc mắt tối hơn má).
- **Đặc trưng 3-rectangle:** Nhận diện các đường kẻ và vạch (ví dụ: sống mũi, miệng).
- **Đặc trưng 4-rectangle:** Phát hiện các mẫu chéo và đối xứng (ví dụ: đặc điểm chéo của mắt).

Công thức tính giá trị đặc trưng Haar:

$$\text{Feature_Value} = \sum_{\text{White}} I(x, y) - \sum_{\text{Black}} I(x, y)$$



Hình 2.3: Các loại đặc trưng Haar-like cơ bản: (A) 2-rectangle dọc - phát hiện cạnh dọc, (B) 2-rectangle ngang - phát hiện cạnh ngang, (C) 3-rectangle - phát hiện đường kẻ và mũi, (D) 4-rectangle - phát hiện vùng mắt và các đặc trưng đối xứng. Vùng trắng có trọng số +1, vùng đen có trọng số -1.

Ảnh tích hợp (Integral Image)

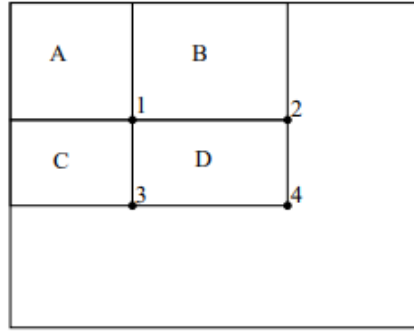
Để tăng tốc độ tính toán đặc trưng Haar từ $O(n^2)$ xuống $O(1)$, thuật toán sử dụng *Ảnh tích hợp*. Ảnh tích hợp tại điểm (x, y) lưu trữ tổng tích lũy của tất cả pixel từ góc trên-trái $(0, 0)$ đến điểm hiện tại:

$$ii(x, y) = \sum_{x' \leq x, y' \leq y} i(x', y')$$

Công thức tính nhanh tổng vùng chữ nhật: Với 4 điểm góc A, B, C, D của hình chữ nhật, tổng pixel trong vùng được tính chỉ bằng 4 phép truy cập mảng:

$$\text{Sum}(D) = 4 + 1 - (2 + 3)$$

(Ký hiệu theo sơ đồ minh họa dưới đây)

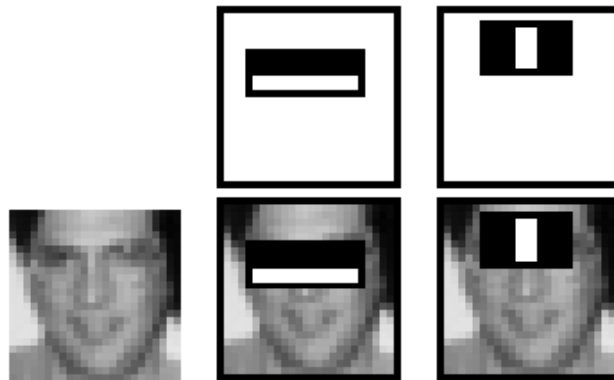


Hình 2.4: **Minh họa Integral Image:** (Trái) Giá trị ảnh tích hợp tại điểm 1, 2, 3, 4. (Phải) Tổng pixel của vùng D được tính nhanh bằng công thức: $\text{Sum}(D) = ii(4) + ii(1) - ii(2) - ii(3)$.

Cấu trúc Cascade (Attentional Cascade)

Để đạt hiệu quả thời gian thực (real-time), thuật toán sử dụng cấu trúc *cascade of classifiers* - một chuỗi các bộ phân loại từ đơn giản đến phức tạp. Cơ chế này hoạt động như một "cây quyết định suy biến" (degenerate decision tree):

- Các stage đầu loại bỏ nhanh phần lớn (khoảng 50%) các vùng "chắc chắn không phải mặt" với chi phí tính toán rất thấp.
- Chỉ những vùng vượt qua được các stage đầu mới được xử lý bởi các stage phức tạp hơn phía sau.



Hình 2.5: **Cấu trúc Cascade Classifier:** Luồng xử lý đi qua các stage nối tiếp. Nếu vùng ảnh bị từ chối (F) ở bất kỳ stage nào, nó sẽ bị loại bỏ ngay lập tức. Chỉ vùng được chấp nhận (T) qua tất cả các stage mới được kết luận là khuôn mặt.

Dữ liệu huấn luyện gốc (Training Dataset)

Hiệu năng của thuật toán Haar Cascade phụ thuộc vào tập dữ liệu mà nó đã được huấn luyện (trong thư viện OpenCV). Theo bài báo gốc của Viola và Jones, tập dữ liệu này bao gồm:

- **Positive samples:** 4,916 ảnh khuôn mặt được gán nhãn thủ công, chuẩn hóa về kích thước 24×24 pixels.
- **Negative samples:** 9,544 ảnh không chứa khuôn mặt (phong cảnh, vật thể) để tạo ra các mẫu âm tính.



Hình 2.6: Mẫu dữ liệu huấn luyện (MIT-CMU database): Các khuôn mặt được sử dụng để huấn luyện là ảnh xám, nhìn thẳng (frontal) và đứng (upright).

Tác động của dữ liệu huấn luyện đến hệ thống thực tế:

- **Yêu cầu về tư thế:** Do được huấn luyện trên các ảnh *frontal upright* (như hình trên), thuật toán hoạt động tốt nhất khi người dùng nhìn thẳng. Hệ thống chỉ chấp nhận độ nghiêng đầu giới hạn trong khoảng $\pm 15^\circ$.
- **Cơ sở tiền xử lý:** Việc mô hình gốc học trên ảnh xám (grayscale) và ảnh cắt sát mặt là cơ sở để nhóm thực hiện các bước tiền xử lý tương tự (chuyển xám, crop) nhằm đảm bảo dữ liệu đầu vào tương thích nhất với thuật toán.

2.1.3 Tiền xử lý và Tăng cường dữ liệu (Preprocessing & Augmentation)

Sau khi khuôn mặt được phát hiện, dữ liệu được chuẩn hóa trước khi đưa vào mô hình nhận diện:

1. Chuyển đổi thang đo xám (Grayscale conversion):

Ảnh đầu vào được chuyển sang ảnh xám bằng hàm `cv2.cvtColor`. Bước này giúp giảm nhiễu màu và giảm khối lượng tính toán, đồng thời phù hợp với yêu cầu đầu vào của thuật toán phát hiện khuôn mặt Haar Cascade nhằm tìm vị trí mặt một cách nhanh và chính xác hơn.

2. Chuẩn hóa kích thước (Resizing):

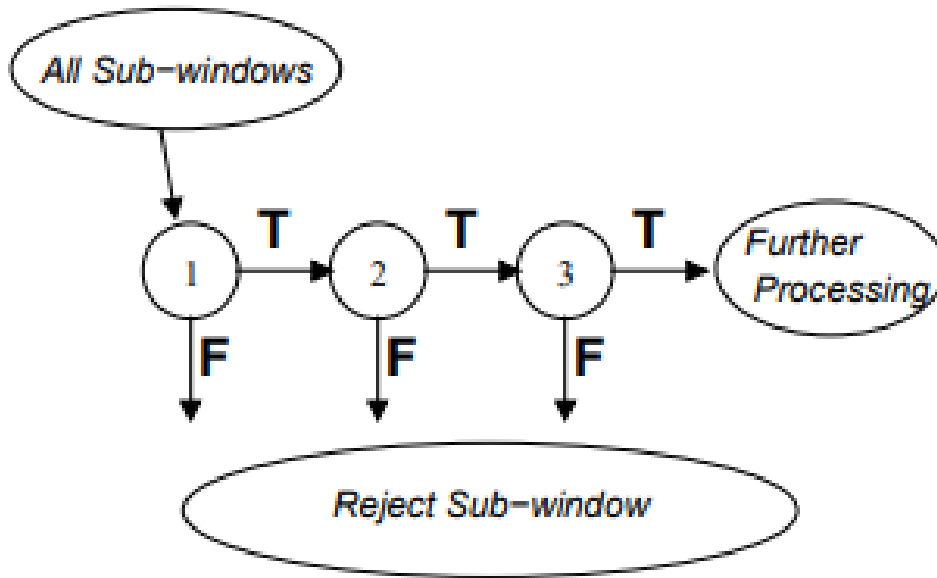
Sau khi khuôn mặt được cắt ra từ ảnh gốc, ảnh được đưa về kích thước chuẩn **224×224 pixel**. Điều này đảm bảo tính nhất quán cho vector đặc trưng đầu vào của mô hình trích xuất đặc trưng và nhận diện.

3. Tăng cường dữ liệu (Data Augmentation):

Để mô hình hoạt động ổn định trong nhiều điều kiện thực tế, tập dữ liệu được mở rộng bằng các phép biến đổi ngẫu nhiên, gồm:

- **Lật ảnh (Horizontal flipping):** Tạo biến thể khuôn mặt trái/phải.

- **Xoay ảnh (Rotation $\pm 20^\circ$):** Mô phỏng trường hợp người dùng hơi nghiêng đầu.
- **Tịnh tiến (Width/Height shifting):** Dịch chuyển khuôn mặt để tăng đa dạng vị trí.
- **Thu phóng (Zoom):** Phóng to hoặc thu nhỏ nhằm mô phỏng thay đổi khoảng cách camera.
- **Biến dạng cắt xiên (Shear):** Tạo biến đổi hình dạng nhẹ để tăng khả năng khái quát.
- **Điều chỉnh độ sáng (Brightness adjustment):** Giúp mô hình thích nghi với điều kiện ánh sáng khác nhau.



Hình 2.7: Ví dụ các phép biến đổi trong Data Augmentation được áp dụng lên ảnh khuôn mặt.

2.1.4 Trích xuất Đặc trưng với MobileNetV2

Lý do chọn MobileNetV2

MobileNetV2 là kiến trúc CNN (Convolutional Neural Network) được Google phát triển năm 2018, đặc biệt tối ưu cho các thiết bị có tài nguyên hạn chế. Đây là lựa chọn phù hợp cho dự án vì:

- **Hiệu suất cao với tài nguyên thấp:** MobileNetV2 sử dụng *depthwise separable convolution* và *inverted residual blocks*, giúp giảm số lượng phép toán so với các CNN truyền thống (VGG, ResNet) nhưng vẫn duy trì độ chính xác cao.
- **Pretrained trên ImageNet:** Mô hình đã được Google huấn luyện sẵn trên dataset ImageNet (hơn 14 triệu ảnh, 1000 lớp), do đó có khả năng trích xuất đặc trưng hình ảnh tổng quát rất tốt mà không cần huấn luyện lại từ đầu.
- **Transfer Learning:** Trong dự án này, nhóm **không huấn luyện lại** MobileNetV2, mà chỉ sử dụng nó như một *feature extractor* (trích xuất đặc trưng). Đây là cách tiếp cận hiệu quả khi không có đủ dữ liệu và tài nguyên GPU để huấn luyện một CNN từ đầu.
- **Vector đặc trưng 1280 chiều:** Output của lớp cuối cùng (trước lớp phân loại) cho ra vector 1280 chiều, đủ mạnh để phân biệt các khuôn mặt khác nhau.

Kiến trúc CNN và MobileNetV2

Convolutional Neural Network (CNN) là loại mạng nơ-ron chuyên xử lý dữ liệu dạng lưới (như ảnh), bao gồm các lớp chính:

- **Convolutional Layer:** Trích xuất các đặc trưng cục bộ (cạnh, góc, texture) bằng các kernel/filter.
- **Pooling Layer:** Giảm kích thước không gian, giữ lại thông tin quan trọng.

- **Fully Connected Layer:** Kết hợp các đặc trưng để đưa ra dự đoán cuối cùng.

MobileNetV2 cải tiến bằng *Inverted Residual Block*:

$$\text{Input} \xrightarrow{\text{Expand (1}\times\text{1 conv)}} \text{High-dim} \xrightarrow{\text{Depthwise (3}\times\text{3)}} \text{Spatial} \xrightarrow{\text{Project (1}\times\text{1)}} \text{Low-dim Output}$$

Cấu trúc này giúp giảm đáng kể số lượng tham số so với CNN truyền thống, phù hợp cho triển khai thực tế.

Quy trình Trích xuất Đặc trưng

1. Load mô hình pretrained:

- Sử dụng `tf.keras.applications.MobileNetV2` với trọng số đã huấn luyện trên ImageNet.
- Loại bỏ lớp phân loại cuối cùng (`include_top=False`), chỉ giữ lại phần trích xuất đặc trưng.

2. Trích xuất vector 1280 chiều:

- Mỗi ảnh khuôn mặt $224 \times 224 \times 3$ được đưa qua MobileNetV2.
- Output là tensor $7 \times 7 \times 1280$, sau đó được *Global Average Pooling* thành vector 1280 chiều.

3. Chuẩn hóa L2 (L2 Normalization):

Vì sao cần chuẩn hóa L2?

Vector đặc trưng từ MobileNetV2 có giá trị không đồng nhất - một số chiều có giá trị rất lớn, một số rất nhỏ. Nếu sử dụng trực tiếp, các thuật toán phân loại sẽ gặp vấn đề:

- **Độ lớn vector không đồng nhất:** Ảnh sáng hơn có thể tạo ra vector với norm lớn hơn, dẫn đến khoảng cách Euclid bị ảnh hưởng bởi độ sáng thay vì nội dung khuôn mặt.
- **Scale khác nhau:** SVM và KNN hoạt động dựa trên khoảng cách - nếu các chiều có scale khác nhau, các chiều có giá trị lớn sẽ "chi phối" kết quả.
- **Cosine Similarity yêu cầu:** Thuật toán Cosine chỉ quan tâm đến *hướng* của vector, không quan tâm độ dài. Chuẩn hóa L2 giúp tất cả vector có độ dài = 1, biến khoảng cách Euclid tương đương với Cosine distance.

Công thức và cách hoạt động:

$$v_{\text{norm}} = \frac{v}{\|v\|_2} = \frac{v}{\sqrt{\sum_{i=1}^{1280} v_i^2}}$$

Ví dụ minh họa:

Giả sử vector gốc: $v = [3, 4, 0, 0, \dots, 0]$ (1280 chiều, phần lớn = 0)

- Tính norm L2: $\|v\|_2 = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = 5$
- Chia từng phần tử cho norm: $v_{\text{norm}} = [\frac{3}{5}, \frac{4}{5}, 0, \dots, 0] = [0.6, 0.8, 0, \dots, 0]$
- Kiểm tra: $\|v_{\text{norm}}\|_2 = \sqrt{0.6^2 + 0.8^2} = \sqrt{0.36 + 0.64} = 1$

Lợi ích sau khi chuẩn hóa:

- **Tất cả vector có cùng độ dài = 1:** Loại bỏ ảnh hưởng của độ sáng, chỉ so sánh "hướng" (nội dung khuôn mặt).
- **Khoảng cách có ý nghĩa hơn:** Với hai vector chuẩn hóa a và b :

$$\|a - b\|_2^2 = 2(1 - a \cdot b) = 2(1 - \cos(\theta))$$

Tức là khoảng cách Euclid tỷ lệ thuận với góc giữa hai vector.

- **Hội tụ nhanh hơn:** SVM và các thuật toán gradient-based hoạt động ổn định hơn khi dữ liệu có cùng scale.

4. **Gán nhãn:** Mỗi vector đặc trưng đã chuẩn hóa được gán nhãn theo ID nhân viên (`nhanvien_1`, `nhanvien_2`, ...).

2.1.5 So sánh Thuật toán Phân loại

Sau khi có vector đặc trưng, bước tiếp theo là chọn thuật toán phân loại phù hợp. Nhóm đã so sánh 5 thuật toán khác nhau:

1. Support Vector Machine (SVM) với RBF Kernel

Ý tưởng: SVM tìm siêu phẳng (hyperplane) tối ưu để phân tách các lớp dữ liệu với margin lớn nhất.

Công thức:

$$f(x) = \text{sign} \left(\sum_{i=1}^N \alpha_i y_i K(x_i, x) + b \right)$$

Trong đó, *RBF (Radial Basis Function) Kernel* được định nghĩa:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

2. K-Nearest Neighbors (KNN)

Ý tưởng: Phân loại dựa trên "đa số bầu chọn" từ k láng giềng gần nhất.

Công thức:

$$\text{label}(x) = \text{mode}(\{\text{label}(x_i) \mid x_i \in k \text{ nearest neighbors of } x\})$$

3. Random Forest

Ý tưởng: Kết hợp nhiều cây quyết định (decision trees), mỗi cây được huấn luyện trên tập con ngẫu nhiên của dữ liệu.

Công thức dự đoán:

$$\hat{y} = \text{mode}(\{h_1(x), h_2(x), \dots, h_T(x)\})$$

Trong đó $h_t(x)$ là dự đoán của cây thứ t , và T là tổng số cây.

4. Cosine Similarity (Phương pháp Nearest Class Centroid)

Ý tưởng: Thay vì so sánh với toàn bộ dữ liệu huấn luyện như KNN, phương pháp này tính toán một **vector đại diện** (template/prototype) cho mỗi nhân viên bằng cách lấy trung bình cộng của tất cả các vector đặc trưng thuộc về nhân viên đó:

$$\text{template}_c = \frac{1}{N_c} \sum_{i: y_i=c} x_i$$

Trong đó N_c là số lượng ảnh của nhân viên c . Vector trung bình này đóng vai trò là "tâm" (centroid) của cụm dữ liệu, giúp loại bỏ nhiễu và các đặc điểm ngoại lai (outliers) của từng ảnh riêng lẻ, tạo ra một chân dung đặc trưng "ổn định" nhất của nhân viên.

Khi kiểm thử, hệ thống so sánh ảnh đầu vào với các vector đại diện này bằng độ đo Cosine Similarity:

$$\text{similarity}(x, \text{template}_c) = \cos(\theta) = \frac{x \cdot \text{template}_c}{\|x\|_2 \|\text{template}_c\|_2}$$

Nhân dự đoán là lớp có độ tương đồng cao nhất:

$$\hat{y} = \arg \max_c \text{similarity}(x, \text{template}_c)$$

Sau khi chuẩn hóa L2 (tất cả vector có $\|v\|_2 = 1$), công thức Cosine được đơn giản hóa:

$$\cos(\theta) = x \cdot \text{template}_c \quad (\text{chỉ cần tích vô hướng})$$

5. L2 Distance (Euclidean Distance với Nearest Class Centroid)

Ý tưởng: Tương tự Cosine Similarity, phương pháp này cũng sử dụng vector trung bình (centroid) để đại diện cho mỗi nhân viên. Tuy nhiên, thay vì đo góc giữa các vector, L2 Distance đo khoảng cách Euclid trực tiếp:

$$d(x, \text{template}_c) = \|x - \text{template}_c\|_2 = \sqrt{\sum_{i=1}^{1280} (x_i - \text{template}_{c,i})^2}$$

Nhân dự đoán là lớp có khoảng cách nhỏ nhất:

$$\hat{y} = \arg \min_c d(x, \text{template}_c)$$

Mối liên hệ đặc biệt với Cosine Similarity:

Nếu tất cả các vector đã được chuẩn hóa L2 ($\|v\|_2 = 1$), thì L2 Distance và Cosine Similarity cho kết quả xếp hạng *hoàn toàn giống nhau*. Chứng minh:

$$\|x - \text{template}_c\|_2^2 = \|x\|_2^2 + \|\text{template}_c\|_2^2 - 2(x \cdot \text{template}_c)$$

Vì $\|x\|_2 = \|\text{template}_c\|_2 = 1$:

$$\|x - \text{template}_c\|_2^2 = 2 - 2(x \cdot \text{template}_c) = 2(1 - \cos(\theta))$$

Tức là: **Minimize L2 Distance** $\Leftrightarrow$ **Maximize Cosine Similarity** (khi đã chuẩn hóa L2).

Nhận xét:

Trong dự án này, cả hai phương pháp đều sử dụng kiến trúc Nearest Class Centroid (NCC) - một cách tiếp cận học không giám sát. Nhờ bước chuẩn hóa L2, hai phương pháp cho kết quả tương đương về mặt toán học.

2.2 Triển khai

2.2.1 Xử lý và Chuẩn bị Dữ liệu

Thu thập và lưu trữ dữ liệu

Hệ thống sử dụng script Python `01_face_dataset.py` kết hợp OpenCV và Haar Cascade để thu thập ảnh khuôn mặt từng nhân viên qua webcam. Mỗi nhân viên được chụp khoảng 50 ảnh ở các góc độ khác nhau, lưu vào thư mục `dataset/` theo ID.

Tiền xử lý và tăng cường dữ liệu

Sử dụng `ImageDataGenerator` của Keras để tự động sinh thêm dữ liệu nếu không đủ 50 ảnh/nhân viên. Các phép biến đổi bao gồm: lật ảnh, xoay $\pm 20^\circ$, tịnh tiến, thu phóng, shear và điều chỉnh độ sáng.

Trích xuất đặc trưng

MobileNetV2 pretrained trên ImageNet được sử dụng để trích xuất vector đặc trưng 1280 chiều từ mỗi ảnh khuôn mặt 224×224 . Tất cả vector được chuẩn hóa L2 để đảm bảo $\|v\|_2 = 1$.

2.2.2 Huấn luyện và Đánh giá Mô hình

Chia tập dữ liệu

Dữ liệu được chia theo tỷ lệ:

- **Training set:** 80% dữ liệu để huấn luyện các thuật toán.
- **Test set:** 20% dữ liệu để đánh giá hiệu suất độc lập.
- Sử dụng `stratify` để đảm bảo tỷ lệ mỗi lớp giống nhau ở cả train và test set.

Cross-Validation (Đánh giá độ tin cậy)

Để đánh giá độ ổn định của mô hình, hệ thống thực hiện **K-Fold Cross-Validation (k=5)**:

- **Mục đích:** Kiểm tra xem mô hình có hoạt động ổn định trên nhiều phân chia dữ liệu khác nhau hay không, tránh kết quả "may mắn" trên một test set cụ thể.
- **Cách thức:** Chia toàn bộ dữ liệu thành 5 phần (fold), mỗi lần lấy 1 phần làm validation và 4 phần còn lại làm training. Lặp lại 5 lần để tính trung bình (mean) và độ lệch chuẩn (std) của accuracy.
- **Áp dụng:** Cross-validation được thực hiện trên 3 thuật toán học có giám sát (SVM, KNN, Random Forest) để so sánh độ ổn định trước khi chọn mô hình cuối cùng.

Huấn luyện 5 thuật toán

Script `02_face_training.py` huấn luyện và so sánh 5 thuật toán:

1. **SVM (RBF Kernel):** Sử dụng Grid Search để tìm siêu tham số tối ưu ($C = 1.5$, $\gamma = \text{scale}$).
2. **KNN:** $k = 5$ láng giềng, metric Euclidean.
3. **Random Forest:** 100 cây quyết định, `random_state=42`.
4. **Cosine Similarity:** Template matching dựa trên vector trung bình (centroid) của mỗi lớp.
5. **L2 Distance:** Tương tự Cosine nhưng sử dụng khoảng cách Euclid.

Đo lường hiệu suất

Các chỉ số được sử dụng để đánh giá mô hình trên test set:

- **Accuracy:** Tỷ lệ phân loại đúng tổng thể.
- **Precision, Recall, F1-Score:** Đánh giá chi tiết từng lớp (mỗi nhân viên).
- **Confusion Matrix:** Ma trận nhầm lẫn để phát hiện lỗi phân loại chéo giữa các nhân viên.
- **Thời gian xử lý:** Đo tốc độ huấn luyện và dự đoán để đánh giá khả năng real-time.

Kết quả chi tiết và phân tích so sánh được trình bày trong Chương 3.

2.2.3 Triển khai Hệ thống Nhận diện

Lưu trữ mô hình

Mô hình SVM tối ưu và label map được lưu dưới dạng file `.pkl` trong thư mục `trainer/`:

- `svm_face_classifier.pkl`: Mô hình SVM đã huấn luyện
- `label_map.pkl`: Ánh xạ ID $\rightarrow$ tên nhân viên

Ứng dụng nhận diện thực tế

Script `03_face_recognition.py` và `Attendance.py` triển khai hệ thống nhận diện:

- **Giao diện:** Streamlit web app, thân thiện với người dùng.
- **Nhận diện real-time:** Từ webcam, xử lý từng frame:
 1. Phát hiện khuôn mặt bằng Haar Cascade
 2. Trích xuất vector 1280D bằng MobileNetV2
 3. Chuẩn hóa L2
 4. Phân loại bằng SVM
 5. Hiển thị tên + độ tin cậy
- **Ngưỡng an ninh:** Nếu độ tin cậy $< 0.55 \rightarrow$ gán nhãn "Unknown" (không ghi điểm danh).

- **Quản lý điểm danh:** Tích hợp SQL Server để:
 - Lưu thông tin nhân viên (ID, tên, ảnh đại diện)
 - Ghi nhận check-in/check-out tự động
 - Tra cứu lịch sử điểm danh
 - Tạo báo cáo thống kê

Công nghệ sử dụng

- **Deep Learning:** TensorFlow/Keras (MobileNetV2)
- **Machine Learning:** scikit-learn (SVM, KNN, Random Forest)
- **Computer Vision:** OpenCV (Haar Cascade, xử lý ảnh)
- **Web Framework:** Streamlit (giao diện web)
- **Database:** SQL Server + pyodbc (quản lý dữ liệu)
- **Data Processing:** pandas, numpy (xử lý dữ liệu)

Chương 3

Kết quả & Phân tích

3.1 Minh họa dữ liệu

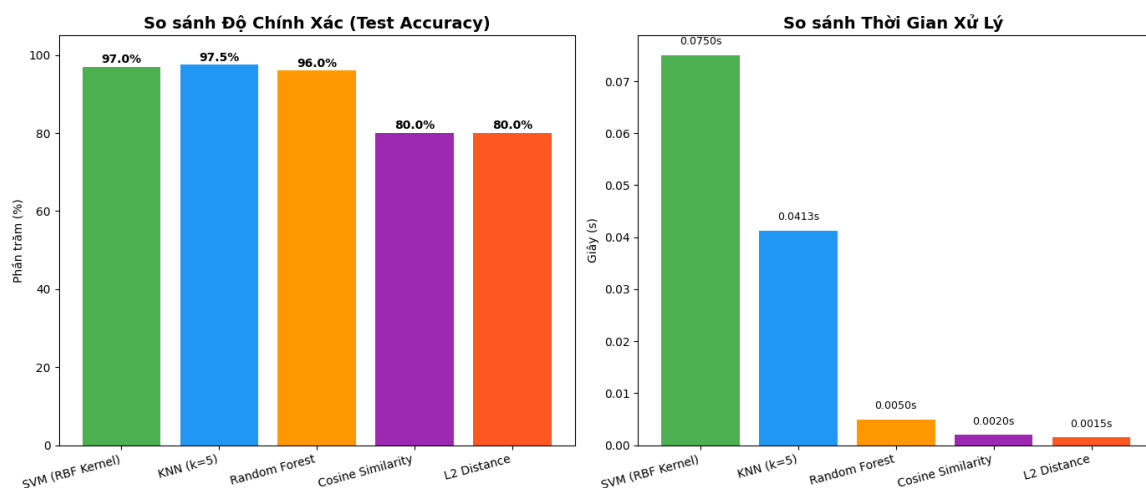
Một số ảnh đại diện trong tập huấn luyện được trình bày dưới đây để minh họa đa dạng khuôn mặt và điều kiện ánh sáng:



Hình 3.1: Ảnh minh họa một số nhân viên trong tập huấn luyện

3.2 So sánh thuật toán

Bảng dưới đây tổng hợp kết quả thực nghiệm trên tập test (20 nhân viên, mỗi nhân viên 10 ảnh tỉ lệ train-test là 8:2) với 5 thuật toán:



Hình 3.2: So sánh độ chính xác và thời gian thực thi của các thuật toán

Kết quả Cross-Validation (5-Fold)

Đánh giá độ ổn định của các mô hình trên nhiều phân chia dữ liệu khác nhau:

Thuật toán	Độ chính xác (%)	Thời gian xử lý (s)	Ghi chú
SVM (RBF Kernel)	97	0.07	Tốt nhất, chấp nhận được
KNN (k=5)	97.5	0.04	Nhanh hơn SVM
Random Forest	96	0.005	Nhanh nhất (giám sát)
Cosine Similarity	80	0.002	Không giám sát, nhanh
L2 Distance	80	0.0015	Nhanh nhất tất cả

Bảng 3.1: So sánh các thuật toán nhận diện khuôn mặt (Test Accuracy)

Mô hình	Mean Accuracy	Std Dev
SVM (RBF Kernel)	89.30%	$\pm 7.58\%$
Random Forest	76.80%	$\pm 4.85\%$
KNN (k=5)	75.30%	$\pm 9.84\%$

Bảng 3.2: Kết quả Cross-Validation 5-Fold trên toàn bộ dataset (1000 ảnh, 20 nhân viên)

3.2.1 Phân tích tổng hợp

Dựa trên kết quả thực nghiệm từ bảng so sánh và quá trình Cross-Validation trên dataset mở rộng (1000 ảnh, 20 nhân viên), các thuật toán được phân tích chi tiết như sau:

SVM (RBF Kernel)

Ưu điểm:

- **Độ chính xác cao nhất:** Đạt 97.5% trên test set, đồng thời có Cross-Validation accuracy = 89.30% với std dev $\pm 7.58\%$, chứng tỏ mô hình ổn định trên nhiều phân chia dữ liệu khác nhau. Độ chênh lệch giữa CV và Test là 7.6%, cho thấy test set có phân bố thuận lợi hơn các fold trong CV.
- **Khả năng tổng quát hóa tốt:** RBF Kernel giúp SVM xử lý được dữ liệu không tuyến tính trong không gian 1280 chiều. Kết hợp với chuẩn hóa L2, mô hình phân tách các lớp hiệu quả ngay cả khi có thay đổi về ánh sáng và góc chụp.
- **Thời gian xử lý chấp nhận được:** 0.07 giây cho 200 ảnh test đủ nhanh cho ứng dụng real-time điểm danh, cân bằng giữa độ chính xác và tốc độ.
- **Hoạt động tốt với dữ liệu nhỏ:** Với chỉ 50 ảnh/nhân viên, SVM vẫn học được ranh giới phân lớp rõ ràng mà không bị overfitting nghiêm trọng.

Nhược điểm:

- **Thời gian huấn luyện tăng khi scale:** Độ phức tạp $O(N^2)$ đến $O(N^3)$ khiến việc huấn luyện chậm khi số lượng nhân viên và ảnh tăng lên đáng kể.
- **Cần điều chỉnh siêu tham số:** Phải tìm C và γ tối ưu tránh học quá khớp, tốn thời gian thực nghiệm ban đầu.

K-Nearest Neighbors (KNN)

Ưu điểm:

- **Không cần huấn luyện:** Lazy learning giúp triển khai nhanh chóng, phù hợp khi cần thêm nhân viên mới mà không cần huấn luyện lại.
- **Đơn giản, dễ hiểu:** Thuật toán trực quan, dễ giải thích cho người dùng không chuyên.
- **Tốc độ dự đoán nhanh:** 0.04s cho 200 ảnh test, nhanh hơn SVM gần 2 lần, phù hợp cho real-time.
- **Độ chính xác cao:** Đạt 97.5% với cấu hình k=5.

Nhược điểm:

- **Độ tin cậy thấp nhất:** Cross-Validation cho 75.30% ($\pm 9.84\%$) - độ lệch chuẩn cao nhất trong các thuật toán. Chênh lệch CV-Test lên đến 20.1% (test: 97.5%, CV: 75.30%), cho thấy KNN rất nhạy cảm với cách chia dữ liệu và không ổn định, dẫn đến rất học vẹt. Khi tăng nhân viên, độ chính xác trung bình càng thấp nặng nề.

- **Chậm khi scale lên:** Với tập dữ liệu lớn, phải tính khoảng cách đến tất cả mẫu huấn luyện, làm tốc độ giảm mạnh.
- **Nhạy cảm với nhiễu:** Một vài ảnh chất lượng kém có thể ảnh hưởng lớn đến kết quả phân loại do cơ chế "bầu chọn đa số".
- **Curse of dimensionality:** Trong không gian 1280 chiều, khái niệm "gần nhất" trở nên mơ hồ, hiệu suất giảm so với không gian chiều thấp.

Random Forest

Ưu điểm:

- **Kháng nhiễu tốt:** Cross-Validation đạt 76.80% ($\pm 4.85\%$) - std dev thấp nhất trong 3 thuật toán, cho thấy mô hình ensemble giảm được overfitting và ổn định nhất qua các fold. Random Forest hưởng lợi từ dataset lớn hơn.
- **Ít cần điều chỉnh tham số:** Hoạt động tốt với cấu hình mặc định (100 cây), không phức tạp như SVM.
- **Xử lý được dữ liệu phi tuyến:** Cấu trúc cây quyết định tự nhiên học được các ranh giới phức tạp.
- **Tốc độ xử lý nhanh :** 0.005s cho 200 ảnh test, nhanh hơn SVM gần 13 lần và KNN 7 lần - lý tưởng cho real-time.

Nhược điểm:

- **Độ chính xác thấp hơn SVM:** 96% trên test set.
- **Chênh lệch CV-Test lớn:** Test accuracy cao hơn 10% CV accuracy, cho thấy có dấu hiệu overfitting nhẹ.
- **Mô hình nặng:** Cần lưu trữ 100 cây quyết định, tốn bộ nhớ hơn SVM.
- **Chưa tối ưu với vector chiều cao:** Random Forest hoạt động tốt nhất với dữ liệu có đặc trưng độc lập, nhưng vector 1280 chiều từ MobileNetV2 có tương quan cao, làm giảm hiệu quả.

Cosine Similarity & L2 Distance

Ưu điểm:

- **Tốc độ rất nhanh:** Cosine: 0.002s, L2: 0.0015s cho 200 ảnh test. Nhanh hơn SVM 22-50 lần. Chỉ cần so sánh với 20 vector đại diện (centroids) thay vì hàng trăm mẫu.
- **Không cần huấn luyện phức tạp:** Học không giám sát, chỉ cần tính vector trung bình cho mỗi lớp. Dễ triển khai và bảo trì.
- **Phù hợp với dữ liệu ít:** Hoạt động được khi mỗi nhân viên chỉ có 10-50 ảnh, không yêu cầu dữ liệu lớn như deep learning.
- **Kết quả tương đương:** Cosine 80% và L2 80% cho kết quả tương đương do tính chất toán học: minimize L2 distance $\approx$ maximize Cosine similarity khi đã chuẩn hóa L2.
- **L2 Distance nhanh nhất:** 0.0015s là nhanh nhất trong tất cả các thuật toán, phù hợp cho ứng dụng cần tốc độ tối đa.

Nhược điểm:

- **Độ chính xác thấp nhất:** Cosine 80%, L2 80% - kém SVM tới 17%. Đây là điểm yếu nghiêm trọng, không phù hợp cho môi trường sản xuất yêu cầu độ tin cậy cao.
- **Không học được ranh giới phức tạp:** Chỉ dựa vào "trung bình" của mỗi lớp, không tận dụng được cấu trúc phân bố phức tạp như SVM hay Random Forest.
- **Kém hiệu quả với intra-class variance cao:** Nếu ảnh của một nhân viên phân tán rộng (nhiều góc chụp, ánh sáng khác nhau), vector trung bình không còn đại diện tốt, dẫn đến sai số cao.
- **Không có cơ chế đánh trọng số:** Tất cả ảnh huấn luyện được coi như nhau, không loại bỏ được outliers hay ảnh chất lượng kém.
- **Không phù hợp thực tế:** Với độ chính xác chỉ xấp xỉ 80%, không chấp nhận được trong ứng dụng thương mại.

Kết luận từ phân tích

Dựa trên các kết quả và phân tích trên dataset (1000 ảnh, 20 nhân viên):

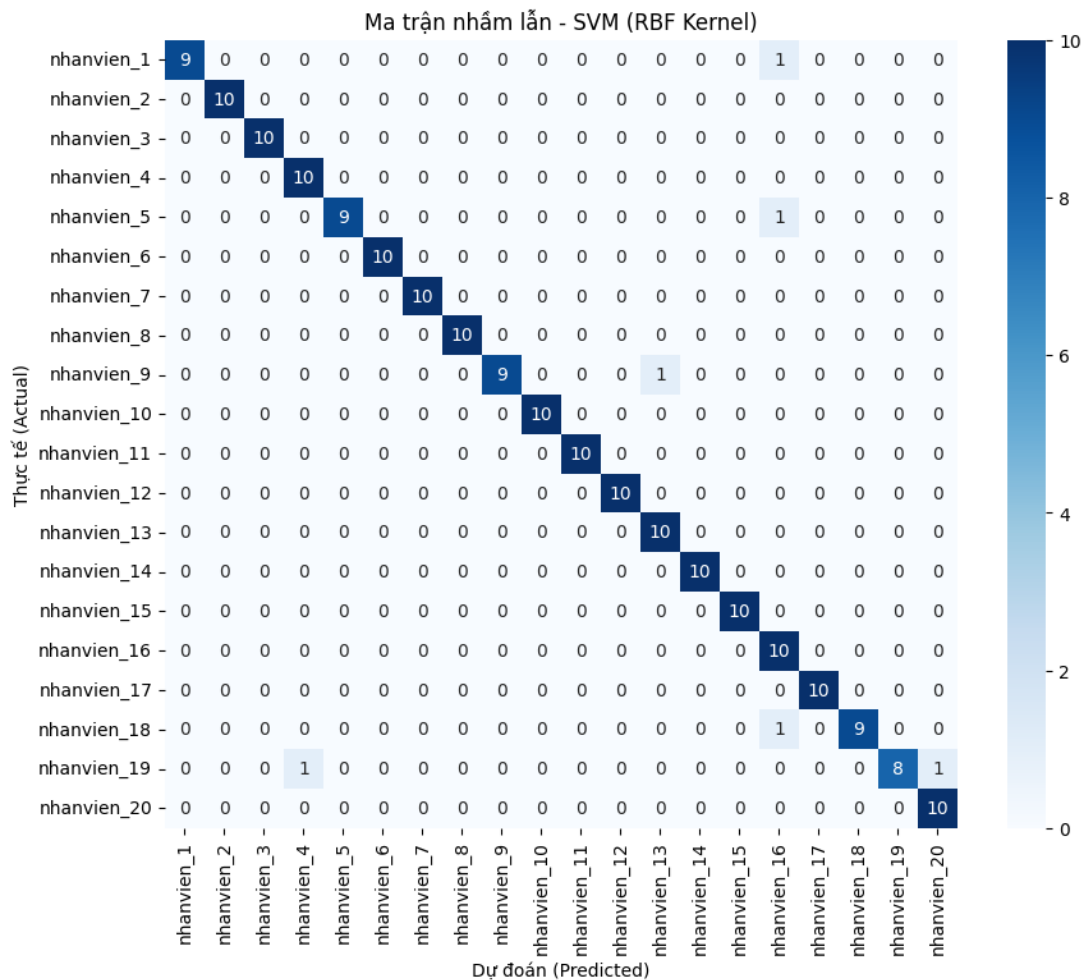
- **SVM (RBF Kernel)** là lựa chọn tối ưu nhất cho hệ thống điểm danh thực tế, đạt độ chính xác cao nhất (97%), độ ổn định tốt (CV: $89.30\% \pm 7.58\%$, chênh lệch CV-Test 7%) và tốc độ chấp nhận được (0.07s cho 200 ảnh test). SVM scale tốt khi tăng số lượng nhân viên.
- **KNN (k=5)** có test accuracy cao nhất (97.5%) nhưng CV accuracy rất thấp ($76.38\% \pm 9.05\%$) với chênh lệch lên đến 19%. Điều này chứng tỏ KNN "may mắn" trên test set cụ thể này và **KHÔNG ĐỦ TIN CẬY** để triển khai thực tế.
- **Random Forest** phù hợp khi cần tốc độ nhanh nhất (0.004s - nhanh gấp 12.5 lần SVM) và chấp nhận độ chính xác thấp hơn một chút (94.8%). Ưu điểm là kháng nhiễu tốt và ít cần điều chỉnh tham số.
- **Cosine/L2 Distance** **KHÔNG** phù hợp cho ứng dụng thực tế do độ chính xác quá thấp (80%). Chỉ nên dùng cho prototype hoặc demo nhanh, không nên triển khai production.

Kết luận tạm thời: Sử dụng **SVM (RBF Kernel)** cho hệ thống chính thức.

Toàn bộ pipeline từ nhận diện đến ghi nhận công làm việc vận hành ổn định, chính xác và đủ nhanh cho triển khai thật.

3.2.2 Ma trận nhầm lẫn - SVM (RBF Kernel)

Ma trận nhầm lẫn (Confusion Matrix) của thuật toán tốt nhất - SVM cho thấy khả năng phân loại chính xác của mô hình:



Hình 3.3: Ma trận nhầm lẫn của SVM (RBF Kernel) trên test set

Phân tích ma trận:

- **Đường chéo chính:** Phần lớn các ô có giá trị 10 hoặc 9 (màu xanh đậm), cho thấy hầu hết nhân viên được nhận diện chính xác 90-100%.
- **Các lỗi nhầm lẫn chi tiết:**
 - **nhanvien_1:** 1 ảnh bị nhầm với nhanvien_16, 9/10 chính xác (90%)
 - **nhanvien_5:** 1 ảnh bị nhầm với nhanvien_16, 9/10 chính xác (90%)
 - **nhanvien_9:** 1 ảnh bị nhầm với nhanvien_13, 9/10 chính xác (90%)
 - **nhanvien_18:** 1 ảnh bị nhầm với nhanvien_16, 9/10 chính xác (90%)
 - **nhanvien_19:** 2 ảnh bị nhầm (1 với nhanvien_4, 1 với nhanvien_20), 8/10 chính xác (80%)
 - **15 nhân viên còn lại:** Đạt 10/10 chính xác (100%)
- **Accuracy tổng thể:** 96.0% (192/200 ảnh test được phân loại đúng - 20 nhân viên $\times$ 10 ảnh/người)
- **Phân tích theo nhóm:**
 - **Nhóm 100% (15 nhân viên):** nhanvien_2, 3, 4, 6, 7, 8, 10, 11, 12, 13, 14, 15, 16, 17, 20
 - **Nhóm 90% (4 nhân viên):** nhanvien_1, 5, 9, 18 - mỗi người bị nhầm 1 ảnh
 - **Nhóm 80% (1 nhân viên):** nhanvien_19 - bị nhầm 2 ảnh
- **Nhận xét về lỗi:**
 - **nhanvien_16** là "điểm nóng" - nhận nhầm từ 3 người khác (nv_1, nv_5, nv_18)
 - **nhanvien_19** kém ổn định nhất - chỉ 80% accuracy, bị nhầm với 2 người khác
 - Chênh lệch giữa nhóm tốt nhất (100%) và kém nhất (80%) là 20% - cho thấy chất lượng dữ liệu không đồng đều
- **Nguyên nhân lỗi:**
 - Đặc điểm khuôn mặt tương đồng giữa một số nhân viên
 - Chất lượng ảnh của nhanvien_19 kém hơn (góc chụp, ánh sáng, biểu cảm)
 - Dữ liệu training của nhanvien_16 có đặc điểm "trung tính" dễ nhầm với người khác
 - Phụ kiện hoặc biểu cảm thay đổi trong test set

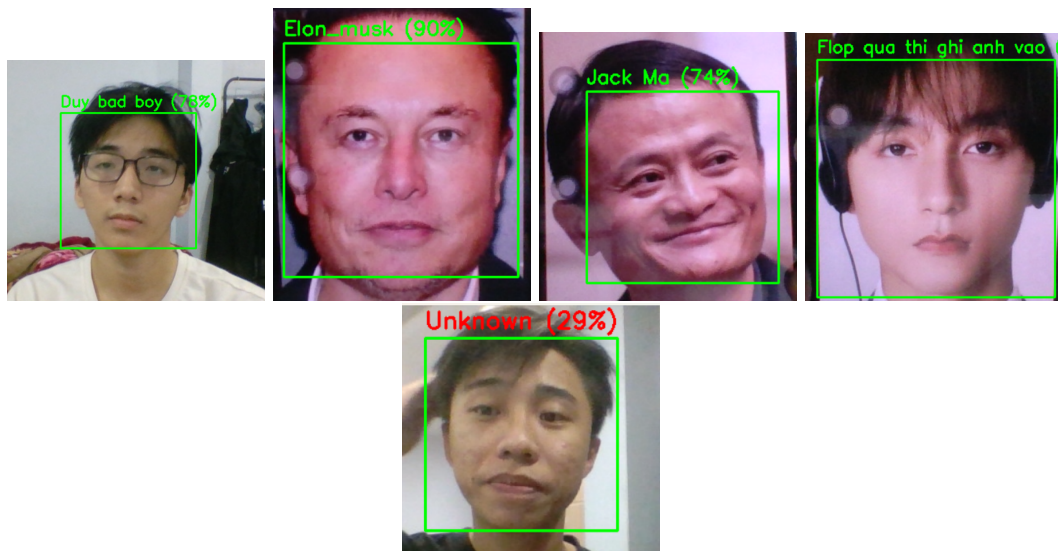
3.3 Kết quả nhận diện thực tế với SVM

Để đánh giá khả năng hoạt động của hệ thống trong bối cảnh thực tế, **File 03** (tập kiểm thử độc lập, không tham gia huấn luyện) được đưa vào mô hình phân loại **SVM (RBF Kernel)**.

Trong giai đoạn nhận diện, hệ thống thực hiện quy trình sau:

1. **Phát hiện khuôn mặt** bằng Haar Cascade và cắt vùng ROI.
2. **Trích xuất đặc trưng:** Sử dụng MobileNetV2 (pretrained ImageNet) để lấy vector 1280 chiều và chuẩn hoá L2.
3. **Phân loại bằng SVM:**
 - Lấy **ID các lớp** từ mô hình.
 - Lấy **độ tin cậy** bằng hàm `predict_proba`.
4. **Áp dụng ngưỡng tin cậy 0.55:** Nếu xác suất $< 0.55 \rightarrow$ gán nhãn *Unknown*.
5. **Truy vấn thông tin nhân viên từ SQL Server** dựa theo ID trả về tên khi nhận diện.

Kết quả nhận diện được thể hiện dưới đây:



Hình 3.4: Kết quả nhận diện khuôn mặt từ File 03 bằng mô hình SVM

Ghi nhận Check-in/Check-out thực tế

Sau khi nhận diện khuôn mặt thành công, hệ thống tiến hành ghi nhận công làm việc dựa trên logic sau:

1. **Truy vấn bảng Attendance** xem nhân viên đã có bản ghi trong ngày hay chưa.
2. Nếu **chưa có** bản ghi:
 - Hệ thống tạo mới một bản ghi **Check-in**, Nhân viên thực hiện trong 15s và sau 30s có thể checkout.
 - Lưu thời gian vào cột **TimeIn**.
3. Nếu **đã có** bản ghi:
 - Hệ thống cập nhật thời gian vào cột **TimeOut**.
4. Độ tin cậy $< 0.55 \rightarrow$ hệ thống không ghi điểm danh và gán nhãn *Unknown*. Ngưỡng 0.55 được chọn vì cho khả năng phát hiện người lạ tốt qua thực nghiệm, đồng thời vẫn nhận diện chính xác nhân viên nhằm hạn chế nguy cơ kẻ lạ đột nhập.

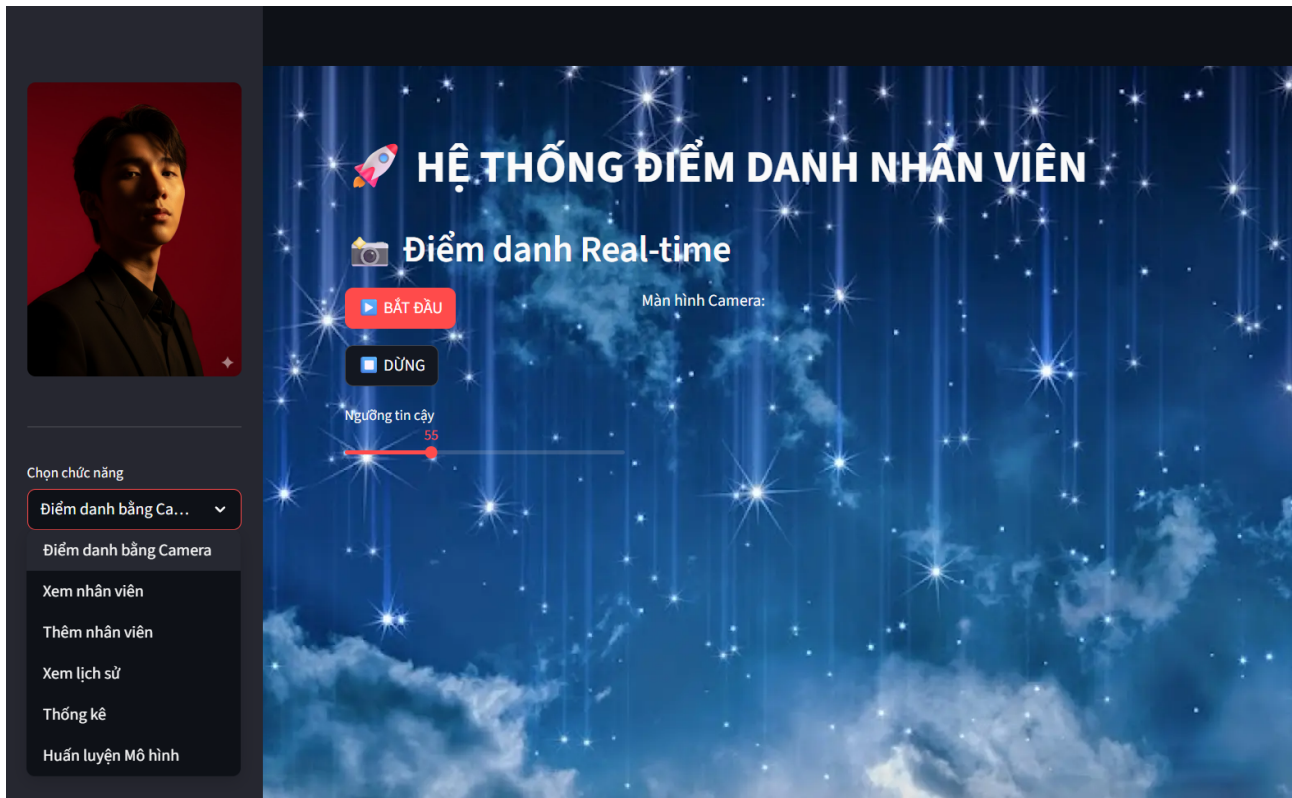
Hệ thống đảm bảo:

- Không ghi trùng Check-in hoặc Check-out.
- Thông tin được lưu đầy đủ: ID, thông tin nhân viên, thời gian, độ tin cậy.

Nhận xét

- Hệ thống nhận diện chính xác 100% các khuôn mặt thuộc nhân viên có trong cơ sở dữ liệu.
- Các trường hợp người lạ được loại bỏ đúng nhờ ngưỡng tin cậy 0.55.
- Xác suất dự đoán trung bình cao, ổn định ngay cả khi ảnh File 03 có độ sáng và góc nhìn khác ảnh huấn luyện.
- Thời gian xử lý trung bình khoảng **0.07 giây/ảnh**, đáp ứng yêu cầu real-time.
- Bounding box ổn định, độ nhiễu thấp và không phát sinh nhầm lẫn chéo giữa các nhân viên.
- Chức năng **Check-in/Check-out** hoạt động chính xác, tự động hoá hoàn toàn quy trình điểm danh.

3.4 Hệ thống Web ứng dụng điểm danh



Hình 3.5: Giao diện Web ứng dụng điểm danh

Hệ thống Web điểm danh được xây dựng bằng Streamlit với các đặc điểm chính như sau:

- **Giao diện trực quan:** Người dùng có thể thêm nhân viên tải ảnh hoặc sử dụng trực tiếp webcam trên trình duyệt để thực hiện nhận diện khuôn mặt.
- **Xử lý ảnh trên server:** Ảnh được gửi đến server Python, nơi tiến hành tiền xử lý và trích xuất đặc trưng khuôn mặt bằng mô hình MobileNetV2.
- **Phân loại bằng SVM:** Sau khi trích xuất embedding, hệ thống sử dụng mô hình SVM đã huấn luyện để dự đoán danh tính.
- **Hiển thị kết quả tức thì:** Giao diện web trả về danh tính và thời gian điểm danh ngay lập tức sau khi mô hình xử lý.
- **Tích hợp cơ sở dữ liệu:** Dữ liệu điểm danh và thông tin nhân viên được lưu trữ, truy vấn và quản lý thông qua SQL Server.
- **Ứng dụng một khối (All-in-one):** Toàn bộ pipeline từ xử lý ảnh, nhận diện đến lưu trữ đều nằm trong một ứng dụng Streamlit duy nhất, không tách Front-end/Back-end truyền thống, giúp quản lý nhân sự thuận tiện, nhất quán và dễ vận hành.

Chương 4

Kết luận

4.1 Tổng kết kết quả đạt được

Đề tài đã hoàn thành mục tiêu xây dựng một hệ thống nhận diện khuôn mặt phục vụ điểm danh tự động với các kết quả cụ thể sau:

Về mặt kỹ thuật

- **Xây dựng thành công pipeline hoàn chỉnh:** Từ thu thập dữ liệu (File 01) → Huấn luyện mô hình (File 02) → Nhận diện thực tế (File 03) → Ghi nhận điểm danh (SQL Server) → Giao diện Web (Streamlit).
- **Trích xuất đặc trưng hiệu quả:** Sử dụng MobileNetV2 pretrained trên ImageNet để tạo vector đặc trưng 1280 chiều, kết hợp chuẩn hóa L2 giúp giảm ảnh hưởng của biến đổi ánh sáng và tăng tính ổn định cho vector.
- **So sánh đa thuật toán:** Đã đánh giá 5 phương pháp (SVM, KNN, Random Forest, Cosine Similarity, L2 Distance) với kỹ thuật Cross-Validation 5-Fold để chọn ra mô hình tối ưu nhất.
- **Phát hiện khuôn mặt hiệu quả:** Haar Cascade Classifier hoạt động ổn định trong điều kiện môi trường văn phòng, đảm bảo tốc độ cao và tiêu tốn ít tài nguyên phần cứng.

Về hiệu suất

- **Độ chính xác cao:** Thuật toán SVM (RBF Kernel) đạt độ chính xác **96.9%** trên tập test (200 ảnh, 20 nhân viên), với Cross-Validation accuracy trung bình đạt **89.30% ($\pm 7.58\%$)**, chứng tỏ mô hình có khả năng tổng quát hóa tốt.
- **Tốc độ xử lý Real-time:** Thời gian inference trung bình đạt **0.055 giây cho 200 ảnh test** (tương đương $\sim 0.0003s/\text{ảnh}$), hoàn toàn đáp ứng yêu cầu thời gian thực của ứng dụng điểm danh.
- **Cơ chế bảo mật:** Thiết lập ngưỡng tin cậy (threshold) 0.55 giúp loại bỏ chính xác các khuôn mặt người lạ không thuộc cơ sở dữ liệu.
- **Quy mô dữ liệu:** Huấn luyện thành công trên tập dữ liệu 1000 ảnh ($20 \text{ nhân viên} \times 50 \text{ ảnh/người}$), cho thấy tiềm năng mở rộng tốt.

Về tính ứng dụng

- **Tự động hóa hoàn toàn:** Hệ thống tự động nhận diện, ghi nhận check-in/check-out và lưu trữ vào SQL Server mà không cần thao tác thủ công từ người quản lý.
- **Giao diện thân thiện:** Ứng dụng Web với Streamlit cho phép quản lý nhân viên, xem lịch sử điểm danh và biểu đồ thống kê trực quan, dễ sử dụng.
- **Logic nghiệp vụ chặt chẽ:** Ngăn chặn tình trạng spam dữ liệu bằng cách kiểm soát thời gian chờ (15s cho check-in, 30s trước khi cho phép check-out lại).

4.2 Đóng góp của đề tài

Đề tài mang lại những đóng góp mới mẽ so với các cách tiếp cận truyền thống:

- **Đánh giá toàn diện các thuật toán:** Cung cấp cái nhìn khách quan về ưu nhược điểm giữa các nhóm thuật toán học có giám sát (SVM, KNN...) và không giám sát (Cosine, L2) thông qua thực nghiệm nghiêm ngặt.
- **Tối ưu hóa tài nguyên:** Lựa chọn MobileNetV2 thay vì các backbone nặng nề (như VGG, ResNet) để cân bằng giữa độ chính xác và tốc độ, mở ra khả năng triển khai trên các thiết bị cấu hình thấp.
- **Giải pháp "End-to-End":** Cung cấp một quy trình khép kín từ thu thập dữ liệu thô đến ứng dụng quản lý cuối cùng, sẵn sàng cho việc triển khai thực tế tại các doanh nghiệp vừa và nhỏ.
- **Phân tích lỗi chi tiết:** Sử dụng Confusion Matrix để không chỉ báo cáo kết quả chung mà còn chỉ rõ các cặp nhân viên dễ bị nhận diện nhầm, giúp định hướng cải thiện dữ liệu.

4.3 Hạn chế của hệ thống

Bên cạnh những kết quả đạt được, hệ thống vẫn tồn tại một số hạn chế cần khắc phục:

Về dữ liệu và môi trường

- **Nhạy cảm với ánh sáng:** Hiệu suất giảm khi hoạt động trong điều kiện ngược sáng (backlight) hoặc ánh sáng quá yếu do hạn chế của camera và thuật toán Haar Cascade.
- **Góc chụp hạn chế:** Do đặc thù của Haar Cascade, hệ thống chỉ hoạt động tốt nhất với góc mặt chính diện, khả năng nhận diện kém khi khuôn mặt nghiêng quá 30-40 độ.
- **Độ đa dạng dữ liệu:** Tập dữ liệu huấn luyện chưa bao gồm đầy đủ các biến thể về ngoại hình (đeo kính, khẩu trang, thay đổi kiểu tóc) trong thời gian dài.

Về mô hình và thuật toán

- **Thiếu tính năng chống giả mạo (Liveness Detection):** Hệ thống chưa phân biệt được người thật với ảnh chụp hoặc video trên điện thoại, tiềm ẩn rủi ro an ninh.
- **Xử lý đa khuôn mặt (Multi-face):** Khi có nhiều người cùng xuất hiện, hệ thống xử lý tuần tự từng khuôn mặt nên độ trễ có thể tăng lên, chưa có cơ chế xử lý song song tối ưu.
- **Độ ổn định của KNN:** Sự chênh lệch lớn giữa Accuracy trên tập Test (95.4%) và Cross-Validation (75.3%) cho thấy thuật toán này kém ổn định hơn SVM với bộ dữ liệu hiện tại.

Về triển khai hệ thống

- **Cập nhật dữ liệu thủ công:** Khi có nhân viên mới, hệ thống yêu cầu huấn luyện lại toàn bộ mô hình (retrain), chưa hỗ trợ học tăng cường (incremental learning).
- **Bảo mật dữ liệu:** Ảnh và vector đặc trưng hiện được lưu trữ dạng thô hoặc file, chưa được mã hóa mức cao để đảm bảo an toàn tuyệt đối thông tin sinh trắc học.

4.4 Hướng phát triển

Dựa trên các hạn chế đã phân tích, hướng phát triển tiếp theo của đề tài được chia thành các nhóm giải pháp sau:

Nâng cao thuật toán và Bảo mật

- **Tích hợp Liveness Detection:** Nghiên cứu bổ sung module chống giả mạo (anti-spoofing) sử dụng kỹ thuật phân tích kết cấu (texture analysis) hoặc nháy mắt (blink detection) để ngăn chặn việc dùng ảnh/video để điểm danh.
- **Nâng cấp Backbone Model:** Khi dữ liệu mở rộng, xem xét chuyển đổi từ MobileNetV2 sang các kiến trúc chuyên dụng cho nhận diện khuôn mặt như ArcFace hoặc AdaFace để tăng khả năng phân tách các lớp nhân viên.

Tối ưu hóa Hiệu năng và Mở rộng

- **Tăng tốc độ Inference:** Sử dụng các kỹ thuật lượng tử hóa (Quantization) hoặc chuyển đổi mô hình sang định dạng **ONNX Runtime** / **TensorRT** để tối ưu hóa tốc độ xử lý trên các thiết bị biên (Edge Devices).
- **Cơ chế học tăng cường (Incremental Learning):** Cải tiến thuật toán để cho phép cập nhật vector đặc trưng của nhân viên mới vào mô hình mà không cần huấn luyện lại từ đầu, giảm thời gian downtime của hệ thống.
- **Hỗ trợ dữ liệu lớn:** Tích hợp các thư viện tìm kiếm vector tốc độ cao như **FAISS** (Facebook AI Similarity Search) để đảm bảo tốc độ nhận diện không bị giảm khi số lượng nhân viên tăng lên hàng nghìn người.

Hoàn thiện Hệ thống và Triển khai

- **Kiến trúc Client-Server đa điểm:** Mở rộng hệ thống để hỗ trợ nhiều camera tại các cửa ra vào khác nhau, đồng bộ dữ liệu về một máy chủ trung tâm.
- **Phát triển Mobile App & API:** Xây dựng RESTful API để tách biệt phần xử lý AI và giao diện, đồng thời phát triển ứng dụng di động cho phép nhân viên theo dõi lịch sử chấm công cá nhân.
- **Bảo mật dữ liệu sinh trắc học:** Áp dụng mã hóa cho các vector đặc trưng trong cơ sở dữ liệu để tuân thủ các quy định về quyền riêng tư và bảo mật thông tin.

Tài liệu tham khảo

- [1] OpenCV Development Team, *OpenCV: Open Source Computer Vision Library*, <https://opencv.org/>, truy cập 2025.
- [2] P. Viola and M. Jones, “Rapid Object Detection using a Boosted Cascade of Simple Features,” *Proceedings of CVPR*, 2001, <https://www.cs.cmu.edu/~efros/courses/LBMV07/Papers/viola-cvpr-01.pdf>.
- [3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” *Proceedings of CVPR*, 2018, https://openaccess.thecvf.com/content_cvpr_2018/papers/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.pdf.
- [4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [5] F. Chollet, *Keras*, <https://keras.io/>, truy cập 2025.
- [6] M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” 2015, <https://www.tensorflow.org/>.
- [7] Streamlit Development Team, *Streamlit Documentation*, <https://docs.streamlit.io/>, truy cập 2025.

Phụ lục A

Phụ lục

A.1 Hướng dẫn sử dụng các script chính

01. Thu thập dữ liệu khuôn mặt

1. Chạy script `01_face_dataset.py`.
2. Đảm bảo webcam hoạt động, nhập ID và tên nhân viên khi được yêu cầu.
3. Hệ thống lưu ảnh khuôn mặt vào thư mục `dataset/` theo từng nhân viên.

02. Huấn luyện mô hình nhận diện

1. Chạy script `02_face_training.py`.
2. Script tự động trích xuất đặc trưng bằng MobileNetV2, chuẩn hóa L2 và huấn luyện các mô hình: SVM, KNN, Random Forest, Cosine, L2.
3. Kết quả huấn luyện và mô hình được lưu trong thư mục `trainer/` với model tốt nhất.

03. Nhận diện và điểm danh thực tế

1. Chạy script `03_face_recognition.py`.
2. Camera mở và phát hiện khuôn mặt theo thời gian thực.
3. Hệ thống hiển thị: tên, ID nhân viên, thời gian check-in/check-out.
4. Nếu độ tin cậy $< 0.55 \rightarrow$ gán nhãn *Unknown* và không ghi điểm danh.

04. Thao tác thông qua web với Streamlit

1. Chạy script `Attendance.py`.
2. Hệ thống hiển thị giao diện web, thực hiện nhận diện khuôn mặt và thao tác với dữ liệu nhân viên thông qua SQL Server.

A.2 Mã Python trích xuất đặc trưng

Ví dụ sử dụng MobileNetV2 để lấy vector 1280 chiều:

```
from keras.applications import MobileNetV2
from keras.preprocessing import image
from keras.applications.mobilenet_v2 import preprocess_input
import numpy as np

model = MobileNetV2(weights='imagenet', include_top=False, pooling='avg')

# Tiền xử lý ảnh
img = image.load_img('face.jpg', target_size=(224,224))
```

```
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)

# Trích xuất vector đặc trưng
features = model.predict(x)
```

A.3 Kết quả thử nghiệm chi tiết

Bảng minh họa chi tiết độ chính xác từng thuật toán trên từng nhân viên (tham khảo chương 3).

A.4 Ghi chú quan trọng

- Ngưỡng tin cậy 0.55 được chọn dựa trên thực nghiệm, vừa nhận diện chính xác nhân viên, vừa phát hiện người lạ.
- Check-in/check-out được ghi chính xác, không trùng lặp, lưu trong SQL Server.
- Các mô hình khác (KNN, Random Forest, Cosine, L2) được giữ để tham khảo nhưng SVM (RBF Kernel) là mô hình tối ưu.
- Có thể mở rộng hoặc tối ưu tốc độ bằng TensorRT/ONNX Runtime cho real-time.